

למידה לא מפוקחת

האם אפשר ללמוד לעשות ייצוג של הידע גם בלי תוויות?
בתור בני אדם, אנחנו לומדים שפה ע"י שמיעה גם בלי שנותנים לנו label. אנחנו מוצאים תבניות בנתונים גם בלי התיוג.

הגדרה: בהינתן $\mathcal{D} = \{x_1, \dots, x_n\}; x_i \in \mathbb{R}^d$ דוגמאות i.i.d, למידה לא מפוקחת מוצאת מבניות בנתונים.

יש כמה סוגים של למידה לא מפוקחת:

- למצוא את ההתפלגות שייצרה את הנתונים.
- להוריד את מספר המימדים
- לחלק את המרחב של N הדוגמאות למספר אזורים ככה שהנקודות באותו cluster קרובות אחת לשנייה.

יש כל מיני גישות לחלק לcluster. נדבר כאן על prototyp based clustering.

- הרעיון: יש נקודה שמייצגת את cluster ונקראת "prototype".
- הנקודה הזאת היא לא בהכרח אחת מהדגימות.
- הנקודות בכל cluster קרובות לprototype.
- הדגימות הן בעצם spatial noise שמוסף לprototype.

בצורה פורמלית:

- נתון: נתונים $\mathcal{D} = \{x_1, \dots, x_n\}$ כאשר $x_i \in \mathbb{R}^d$
- היפרפרמטר $K \in \mathbb{N}^+$ שמגדיר את מספר הcluster שאמורים לגלות.
- המטרה: למצוא K prototype $\mu^* = \{\mu_1, \dots, \mu_K\}$ כאשר $\mu_i \in \mathbb{R}^d$.
- μ^* ממזער את סכום ריבועי המרחקים של הנקודות ב $\mathcal{D}$ מהprototype שלהם:

$$\mu^* = \arg \min_{\mu} \sum_{i=1}^n \min_{j \in \{1, \dots, K\}} \|x_i - \mu_j\|_2^2$$

- נקודות שייכות לprototype הקרוב ביותר לפי (ריבוע) מרחק אוקלידי:

$$\text{cluster}(x_i) = \arg \min_j \|x_i - \mu_j\|_2^2$$

הבעיה הזאת מוגדרת היטב, אבל מאוד מורכבת חישובית - $\mathcal{O}(n^{Kd+1})$. כלומר גם עבור שני מימדים $K=2$ זה יוצא $\mathcal{O}(n^5)$! במקום זה נרצה יוריסטיקה איטרטיבית. אלגוריתם k-means הוא אלטרנטיבה כזאת.

- האלגוריתם: אתחול: בוחרים K prototype התחלתיים $\mu^0 = \{\mu_1^0, \dots, \mu_K^0\}$.

• כל איטרציה ($t = 1, \dots$) כוללת שני צעדים:

1. צעד E - משייכים כל דגימה x_i לcluster של prototype הקרוב ביותר:

$$\text{cluster}^{(t)}(x_i) = \arg \min_j \|x_i - \mu_j^{t-1}\|_2^2$$

2. צעד M - מחשבים מחדש את prototype:

$$c_j = \{x_i \mid \text{cluster}^{(t)}(x_i) = j\} \quad \mu_j^t = \frac{1}{|c_j|} \sum_{x \in c_j} x$$

• ממשיכים עד להתכנסות - כלומר עד שמתקיים:

– הנקודות של prototype לא השתנו.

– השייך לcluster לא השתנה.

נשים לב שאלו תנאים שקולים.

האם מובטח שהאלגוריתם יתכנס? כן!
יש objective function:

$$J(\mu, r) = \sum_{i=1}^n \sum_{j=1}^K r_{ij} \|x_i - \mu_j\|_2^2$$

כאשר: $\mu_j \in \mathbb{R}^d$ centroid

$r \in \{0, 1\}^{n \times d}$ - שיוכים - $r_{ij} = 1$ אם x_i משייך לcluster j . אחרת $r_{ij} = 0$.

הרעיון של ההוכחה:

• צעד לא יכול להגדיל את J

• $J \geq 0$

• מספר הcluster האפשריים הוא סופי

למה צעד לא יכול להגדיל את J ? צעד E משייך לפי מה שמקטין את הממוצע (כלומר את J) וצעד M בוחר את μ_j להיות הממוצע - שהוא הרי המספר שממזער את סכום המרחקים¹.

סיכום: יתרונות: • מובטח שהוא יתכנס

• מתכנס מאוד מהר

• בעיות: ההתכנסות תלויה באיתחול - יכולים להיות איתחולים שמביאים

אותנו למינימום מקומי. אפשר להתגבר על זה באמצעות ביצוע

מספר נסיונות ולקיחת התוצאה שבה סכום המרחקים של כל

נקודה מהprototype שלה הכי נמוך.

¹אפשר להוכיח ע"י גזירה.

למידת מכונה
89-2511-02

מקליד: עידן אריה
מרצה: ד"ר אורן גליקמן
תאריך: 2026-06-08

-
- בחירת K לא נכון תוביל לתוצאה לא נכונה.
 - לפעמים cluster שנראים לעין (לפחות בדו-מימד) הם לא cluster מבחינת נורמה.
 - במימד גדול, נורמת L_2 מתחילה לאבד משמעות.

מה עושים אם רוצים להיות מסוגלים לתפוס צורות שונות?

- Gaussian Mixture Model - משתמש במטריצות שונות משותפת.
- DBSCAN - ממזער מרחקים בין הנקודות (ולא למרכזים)
- Hierarchical Clustering - ממדל לפי dendrogram (עצים)

Gaussian Mixture Model

אלגוריתם Gaussian Mixture Model הוא אלגוריתם clustering באמצעות שילוב פרמטרים.

- ההנחה: יש כמה קבוצות, שכל אחת מהן יוצרת התפלגות גאוסיאנית.
- העולם קודם "מגריל" לאיזו קבוצה כל דגימה שייכת, ואז מגריל את הדגימה לפי ההתפלגות שלה.

מבחינה פורמלית:

- כרגיל יש לנו $\mathcal{D} = \{x_1, \dots, x_n\}$ ב- $\mathbb{R}^d$, אבל הפעם $\mathcal{D}$ נוצר משילוב (mixture) של K התפלגויות גאוסיאניות שונות.

- ההסתברות השולית של x_i מוגדרת לפי סכום של K ההתפלגויות:

$$P(x_i|\theta) = \sum_{k=1}^K \pi_k \mathcal{N}(x_i|\mu_k, \Sigma_k)$$

- הפרמטרים הם $\theta = (\pi, \mu, \Sigma)$ כאשר:

π_k ההסתברות להיות שייכים להתפלגות k .

μ_k הממוצע של התפלגות k .

Σ_k מטריצת השונות המשותפת של התפלגות k .

- המטרה - למצוא θ שממקסם את המarginal log-likelihood:

$$\arg \max_{\theta} \sum_{i=1}^n \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(x_i|\mu_k, \Sigma_k) \right)$$

גם את זה אי אפשר לפתור אנליטית, וצריך, שוב, אלגוריתם איטרטיבי:

- צעד E (expectation) - בהינתן $\theta^{(t)}$, מחשבים את הresponsibility γ_{ik} - ההסתברות הפוסטריורית של k היה אחראי לייצר את x_i :

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(x_i|\mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_i|\mu_j, \Sigma_j)}$$

- צעד M (maximization):

$$\mu_k^{(t+1)} = \frac{\sum_{i=1}^n \gamma_{ik} x_i}{\sum_{i=1}^n \gamma_{ik}}$$

$$\Sigma_k^{(t+1)} = \frac{\sum_{i=1}^n \gamma_{ik} (x_i - \mu_k^{(t+1)}) (x_i - \mu_k^{(t+1)})^T}{\sum_{i=1}^n \gamma_{ik}}$$

$$\pi_k^{(t+1)} = \frac{1}{n} \sum_{i=1}^n \gamma_{ik}$$

למידת מכונה
89-2511-02

מקליד: עידן אריה
מרצה: ד"ר אורן גליקמן
תאריך: 2026-06-08

-
- תכונות:
- soft clustering - כל נקודה שייכת לכל אחד מהclusters בהסתברות מסויימת.
 - הclusters יכולים להיות גם אליפסות, לא רק מעגלים.
 - האלגוריתם מתכנס רק אסימפטוטית.
 - הסיבוכיות שלו (לכל איטרציה) היא $O(n \cdot K \cdot d^3)$.

Principal Component Analysis (PCA)

האלגוריתמים הקודמים עסקו בclustering ובdensity estimation. הפעם המטרה לעשות dimensionality reduction - כלומר להקטין את מספר המימדים - למצוא פונקציה $f: \mathbb{R}^d \rightarrow \mathbb{R}^k$ כאשר $k \ll d$. ההנחה היא שהנתונים, בהם באמת k -מימדיים - אבל קיבלנו יצוג d -מימדי שלו.

דוגמה: ייצוג של מנייה יכול להיות העליה והירידה שלה בכל יום - כלומר מימד לכל יום. זה מימד מאוד מאוד גדול. אבל PCA יכול להקטין את זה ל-2 מימדים (שהם לא בהכרח בחירה של אחד מהימים, אלא שילוב לינארי של הכל) וזה יאפשר לנו למצוא cluster יתר בקלות.

אפשר גם לעבוד על הציר השני של המטריצה - לייצג כל יום לא באמצעות 500 מספרים (אחד לכל מנייה) אלא למצוא מימד יותר נמוך שייצג מה קרה באותו יום.

אנחנו אמנם מאבדים מידע, אבל זה מאפשר לנו ליעל את המודלים, להציג את המידע בצורה חזותית, להוריד רעש, ובאופן כללי לדחוס את הנתונים. לפני שעושים PCA מנרמלים את הנתונים:

$$x_{i,j} \rightarrow \frac{x_{i,j} - \bar{x}_j}{\sigma_j}$$

כאשר: $\bar{x}_j$ הממוצע של פיצ'ר j :

$$\bar{x}_j = \frac{1}{n} \sum_{i=1}^n x_{i,j}$$

σ_j סטיית התקן של פיצ'ר j :

$$\sigma_j^2 = \frac{1}{n} \sum_{i=1}^n (x_{i,j} - \bar{x}_j)^2$$

לאחר הנרמול אפשר להניח ממוצע 0 ושונות 1.

תזכורת: אם יש לנו מרחב d מימדי ו- $r < d$, אפשר לבחור ווקטאורי עמודה $\{v_1, \dots, v_r\}; v_i \in \mathbb{R}^d$ שיהיו בסיס לתת-מרחב r מימדי.

• מניחים שהבסיס אורתונורמלי² - ואז $V^T V = I_{r \times r}$.

• תת המרחב הוא $\text{span}\{v_1, \dots, v_n\} = \left\{ \sum_j a_j v_j : a_j \in \mathbb{R} \right\}$

• אפשר לייצג את הבסיס הזה בתור מטריצה $V = [v_1 | \dots | v_r] \in \mathbb{R}^{d \times r}$

²אפשר להפוך את הבסיס לאורתונורמלי באמצעות Gram-Schmidt, בהנחה שהווקטורים בת"ל.

למידת מכונה
89-2511-02

מקליד: עידן אריה
מרצה: ד"ר אורן גליקמן
תאריך: 2026-06-08

אפשר "לקודד" (encode) כל וקטור $x \in \text{span}(V)$ באמצעות V^T , ואז "לפענח" (decode) אותו באמצעות V כלומר $VV^T x$ הוא projection של x לתוך V - ואם מלכתחילה $x \in V$ אז $VV^T x = x$.

נשים לב: $V^T V = I_{r \times r}$ אבל $VV^T \neq I_{d \times d}$ כי V לא מכיל בסיס מלא ל- $\mathbb{R}^d$.
אם $x \notin \text{span}(V)$ אז $VV^T x = x' \neq x$.

למידת מכונה
89-2511-02

מקליד: עידן אריה
מרצה: ד"ר אורן גליקמן
תאריך: 2026-06-08

המטרה של PCA היא למצוא בסיס אורתונורמלי ככה שהשחזור של הדגימות ב- $\mathcal{D}$ יהיה כמה שיותר קרוב למקור:

$$\min_{V: V^T V = I} \sum_i \|V V^T x_i - x_i\|_2^2$$

הערה: אם $r = 1$ זה זהה לרגרסייה לינארית, אבל במימדים יותר גבוהים זה לא כי הבסיס הוא יותר מקו אחד.

אם יש לנו מטריצת של n דגימות ממימד d - $X \in \mathbb{R}^{n \times d}$ - ה sample covariance matrix היא:

$$\text{Cov}(X) = \frac{1}{n} X^T X \in \mathbb{R}^{d \times d}$$

$\text{Cov}(x)$ היא:

- סימטרית, ולכן יש לה ווקטורים עצמיים $u_1, \dots, u_d$ ווקטורים עצמיים $\lambda_1, \dots, \lambda_d$ ממשיים.

- היא גם positive semi-definite (PSD) ולכן כל הערכים העצמיים הם לא-שליליים.

- הווקטורים העצמיים הם אורתוגונליים $u_i \cdot u_j = \delta_{i,j}$

כזכור, המטרה של PCA היא למזער את:

$$\min_{V: V^T V = I_{r \times r}} \sum_i \|V V^T x_i - x_i\|_2^2$$

משפט: הפתרון ל-PCA הוא מטריצה V המורכבת מווקטורים $v_1 = u_1, \dots, v_r = u_r$ כלומר r הווקטורים עם הערכים העצמיים הכי גדולים של $\text{Cov}(X)$.

כלומר - כל מה ש-PCA עושה זה לחשב את $\text{Cov}(X)$ ולקחת את r הווקטורים הכי גדולים. מה קיבלנו? בכל נקודה x , אפשר להטיל אותה ממימד d למימד r באמצעות $V^T x$. מה שמפסידים - ממוצע המרחקים - זה סכום הערכים העצמיים שלא השתמשנו בהם:

$$\frac{1}{n-1} \sum_i \|V V^T x_i - x_i\|_2^2 = \sum_{j=r+1}^d \lambda_j$$

כלומר, אם רוצים לבחור לאיזה מימד להוריד - אפשר להסתכל על הערכים העצמיים וכל עוד הם קטנים אפשר להוריד אותם. כלומר לבחור r קטן - אבל כזה שגם $\sum_{j=r+1}^d \lambda_j$ יהיה קטן.

PCA יוצר decorrelation בנתונים. אם הטלה היא $a = V^T x$ והטלת כל הדגימות היא $A = X V$, הממוצע של A הוא אפס בגלל שנירמלנו את X :

$$\sum_i a_i = V^T \sum_i x_i = V^T 0$$

למידת מכונה
89-2511-02

מקליד: עידן אריה
מרצה: ד"ר אורן גליקמן
תאריך: 2026-06-08

השונויות של A היא אלכסונית (עד כדי factor של n):

$$A^T A = V^T X^T X V = V^T \text{Cov}(X) V = V^T V \Lambda V^T V = \Lambda$$

PCA לא רק ממזער את המרחקים - הוא גם ממקסם את השונויות, שכן המימדים שכן נשארים הם אלו בהם השונויות הייתה גדולה.
כמובן - ככל שיותר קטן ככה השונויות יותר קטנה. אנחנו רוצים לא להקטין את השונויות יותר מדי, כדי שהמודל יהיה טוב.
לא חייבים להגביל את עצמנו להורדות מימד לינאריות. אפשר להשתמש גם בפונקציות אחרות.